

ISIS4426 - Desarrollo de Soluciones Cloud

Entrega 2 - Despliegue básico en la nube

Proyecto: Plataforma web de cursos masivos abiertos en línea (MOOC) **Periodo académico:** 2026-20 **Migración de una aplicación tradicional en contenedores a la nube pública**

Contexto

La organización que opera la plataforma MOOC ha decidido realizar su primer despliegue en la nube pública. La aplicación desarrollada en la entrega anterior permite a profesores autorizados crear cursos mediante la jerarquía Curso → Módulo → Unidad → Recurso, y a estudiantes inscribirse, consumir contenidos, presentar quizzes y obtener insignias verificables.

En esta etapa, el equipo deberá trasladar la solución a un proveedor de nube pública, distribuir sus componentes mediante el servicio de máquinas virtuales, incorporar un servicio administrado de bases de datos relacionales y migrar el almacenamiento de objetos de la entrega anterior al servicio administrado propio del proveedor. El despliegue debe preservar las reglas de negocio y permitir medir cómo se comporta la plataforma cuando aumenta la actividad de estudiantes y profesores. El equipo deberá identificar los servicios del proveedor utilizado que satisfacen las capacidades requeridas y documentar su correspondencia con el modelo de este enunciado.

Esta entrega desarrolla una etapa intermedia de su arquitectura: un despliegue básico sobre infraestructura como servicio (IaaS), con capacidad fija de cómputo y persistencia de archivos en el servicio de almacenamiento de objetos. Las adaptaciones frente a la arquitectura objetivo se delimitan en este enunciado.

Objetivos

- **Analizar y ajustar** la arquitectura de la plataforma MOOC para su despliegue inicial en la nube pública, considerando la distribución de responsabilidades, las comunicaciones entre componentes, la persistencia y la seguridad.
- **Aprovisionar y configurar** máquinas virtuales mediante el servicio de cómputo del proveedor para ejecutar el componente web y los workers, conservando el uso de contenedores Docker.
- **Integrar y configurar** un servicio administrado de bases de datos relacionales con PostgreSQL para la persistencia transaccional, trasladando el esquema y los datos necesarios para operar la aplicación.
- **Migrar** el almacenamiento de objetos al servicio administrado propio del proveedor de nube e integrarlo con la aplicación desplegada, preservando la carga directa, el procesamiento asíncrono y el acceso autorizado a los archivos definidos en la entrega anterior.
- **Verificar** los flujos funcionales y los controles de acceso de la plataforma en el entorno desplegado.
- **Medir y analizar** la capacidad de la solución mediante pruebas de carga y estrés, identificando cuellos de botella y oportunidades de evolución con base en evidencia.

Tiempo de dedicación

La entrega se desarrollará durante **una semana y media**, en los equipos conformados para el proyecto y de acuerdo con el cronograma del curso. Cada integrante deberá participar en la implementación, las pruebas, el análisis y la sustentación de la solución.

Lecturas previas

- Enunciado de la plataforma MOOC y retroalimentación de la entrega anterior.
- Material del curso sobre máquinas virtuales, redes, almacenamiento, bases de datos y análisis de capacidad.

- Documentación del proveedor utilizado sobre el servicio de máquinas virtuales, redes virtuales, reglas de firewall, almacenamiento de objetos y el servicio administrado de bases de datos relacionales.
- Documentación de las herramientas utilizadas para Docker Compose, Redis, asynq, FFmpeg y pruebas de carga.

Esquema de evaluación

Los porcentajes siguientes corresponden a la nota de esta entrega.

Componente evaluado	Porcentaje
Despliegue e integración de componentes	50%
Configuración del servicio administrado de bases de datos relacionales	10%
Funcionamiento de la plataforma y configuración de red	10%
Análisis de capacidad: escenario 1	10%
Análisis de capacidad: escenario 2	10%
Documentación de arquitectura	10%
Total	100%

La distribución conserva **70% para la migración y el funcionamiento, 20% para el análisis de capacidad y 10% para la documentación de arquitectura**. Cada criterio debe respaldarse con evidencia reproducible del entorno desplegado.

Recomendaciones y consideraciones

Continuidad con la entrega anterior

La migración se realizará sobre el código de la plataforma MOOC desarrollado previamente, incorporando las correcciones de la retroalimentación recibida. Se conservan el backend en Go como monolito modular, los workers independientes, PostgreSQL, Redis con asynq y la integración con almacenamiento de objetos.

En esta entrega no se espera integrar nuevas funcionalidades ni desarrollar el frontend. El objetivo es avanzar en un primer despliegue cloud de la solución existente. El trabajo se concentrará en aprovisionar y configurar la infraestructura, desplegar los componentes, ajustar sus conexiones y verificar su funcionamiento y capacidad en la nube, manteniendo el alcance funcional de la entrega anterior.

La validación y la sustentación podrán realizarse mediante clientes HTTP, scripts y pruebas automatizadas sobre la API y los procesos asíncronos. Si el equipo ya dispone de un frontend, podrá reutilizarlo, pero su desarrollo o ampliación no será requisito ni criterio de evaluación de esta entrega.

La separación de componentes en máquinas virtuales corresponde al modelo de despliegue; no exige transformar los módulos de negocio en microservicios. Los componentes de aplicación y Redis deben ejecutarse en contenedores. PostgreSQL se ejecutará en el servicio administrado de bases de datos relacionales del proveedor. Los archivos conservarán el modelo de almacenamiento de objetos de la entrega anterior y se migrarán al servicio administrado de almacenamiento de objetos propio del proveedor de nube.

Alcance de infraestructura de esta entrega

- Se utilizarán **dos máquinas virtuales distintas**, denominadas *Web Server* y *Worker Server*, además de una instancia de PostgreSQL en el servicio administrado de bases de datos relacionales y el servicio de almacenamiento de objetos.
- No se implementarán escalado automático, balanceadores de carga, réplicas de aplicación entre máquinas ni mecanismos de alta disponibilidad. El número de máquinas permanecerá fijo durante las pruebas.
- Los originales y derivados se conservarán en el servicio de almacenamiento de objetos, manteniendo la carga directa y el acceso mediante URLs firmadas (o no firmadas) implementados en la entrega anterior.
- Su sistema de mensaje (Redes, RabbitMQ u otro) se desplegará en un contenedor en *Worker Server*, accesible por la red privada. No se requieren servicios administrados de caché o mensajería en esta etapa.

- No se utilizarán CDN, plataformas administradas de ejecución de contenedores ni funciones serverless para sustituir los componentes solicitados.

Migración al servicio de almacenamiento de objetos de la nube

El almacenamiento de objetos utilizado en la entrega anterior deberá migrarse al servicio administrado de almacenamiento de objetos propio del proveedor de nube. El equipo deberá aprovisionar y configurar este servicio, trasladar los archivos originales y derivados necesarios para las pruebas y ajustar los endpoints, credenciales, permisos y parámetros de conexión de la API y los workers.

Se conservarán la organización lógica de los objetos y los flujos de carga directa, procesamiento asíncrono y acceso mediante URLs firmadas ya implementados (si no han sido implementadas por el momento no es necesario). Se verificará la integridad de los archivos migrados y la correspondencia de sus referencias en la base de datos. Las pruebas y la sustentación deberán demostrar el funcionamiento de estos flujos sobre el servicio administrado de la nube.

En esta etapa, el contenido multimedia se servirá directamente desde el almacenamiento de objetos. La distribución mediante CDN se abordará en una etapa posterior.

Los objetivos de escala, disponibilidad, redundancia y recuperación de la especificación general describen la evolución del producto. **En esta entrega se evalúa la capacidad medida de la configuración básica**, sin exigir autoscaling o alta disponibilidad. Los objetivos de desempeño se utilizarán como referencia para cuantificar brechas, y los controles funcionales de integridad y autorización seguirán siendo obligatorios.

Seguridad y configuración

- Las credenciales, llaves privadas y secretos no deben aparecer en el repositorio, las imágenes de contenedor ni las evidencias de la entrega. La configuración sensible se inyectará en ejecución mediante variables de entorno o archivos protegidos externos al repositorio; se entregará un ejemplo sin valores secretos.
- El acceso de usuarios a la plataforma se realizará por HTTPS, conservando las cookies seguras y la protección CSRF de la aplicación. La conexión a la base de datos administrada debe ser privada.
- La API y los workers accederán a Redis y a la base de datos administrada mediante direcciones privadas y reglas de acceso limitadas a los componentes que los necesitan.
- El acceso al almacenamiento de objetos se realizará por HTTPS, con permisos diferenciados por componente. Cuando se utilice un frontend existente, se configurará CORS para permitir las operaciones necesarias desde su origen.

Gestión de costos

- Registrar la región, los tipos de instancia, el almacenamiento aprovisionado y las horas de uso previstas. Incluir en la estimación el volumen de objetos, las operaciones y las transferencias de datos. Presentar los costos con fecha y supuestos, y contrastarlos con el consumo observado durante las pruebas.
- Activar presupuestos y alertas de consumo cuando los permisos de la cuenta lo permitan. Documentar las limitaciones del laboratorio que afecten esta configuración.
- Encender los recursos cuando sean necesarios para implementar, probar o sustentar. Detener una instancia no implica eliminar todos sus costos; deben revisarse también almacenamiento, respaldos y recursos de red asociados.
- Después de registrar las evidencias y cargar la entrega, eliminar la instancia de base de datos administrada del laboratorio, conservando previamente los respaldos o los datos sintéticos y scripts necesarios para reconstruirla. Documentar qué recursos se conservan, sus costos y cómo recrear el entorno para una sustentación.
- Si el servicio de bases de datos permite detener o suspender temporalmente una instancia, consultar las condiciones del proveedor sobre duración, reactivación automática y cargos que continúan durante la suspensión. Incorporar esas condiciones al plan de uso de recursos.

Modelo de despliegue básico en la nube pública

La organización ha definido una configuración inicial de **2 vCPU, 2 GiB de RAM y 30 GiB de almacenamiento persistente por máquina virtual**. El equipo debe identificar el perfil de cómputo y la capacidad de almacenamiento disponibles en su cuenta que correspondan a esta configuración. Si el catálogo del proveedor no ofrece una combinación exacta, deberá seleccionar la más cercana que cubra los recursos indicados, justificarla y registrar la configuración efectiva. Las imágenes de aplicación pueden construirse previamente para reducir el consumo de memoria durante el despliegue.

Si una dependencia no puede ejecutarse con esos recursos, deberá documentarse el fallo, el ajuste mínimo aplicado y su efecto sobre costo y capacidad. La configuración efectiva debe permanecer fija durante cada corrida; cualquier cambio de tamaño exige identificar los resultados como una configuración diferente.

Recurso	Componentes y responsabilidad
Servicio de máquinas virtuales - Web Server	API modular en Go y proxy inverso; podrá alojar el frontend si ya existe.
Servicio de máquinas virtuales - Worker Server	Workers en Go con sus dependencias de procesamiento, junto con asynq y su cola de mensajería.
Servicio de almacenamiento de objetos	Persistencia de originales, derivados HLS, documentos, miniaturas e imágenes.
Servicio administrado de bases de datos relacionales - PostgreSQL	Persistencia de cuentas, estructura académica, recursos y metadatos, inscripciones, intentos, calificaciones, progreso, trabajos, etc. Despliegue en una sola zona de disponibilidad, sin réplicas de lectura.

La capacidad de cómputo y el almacenamiento de la base de datos administrada deberán seleccionarse según la disponibilidad de la cuenta y el presupuesto, y registrarse en el informe. Los límites anteriores corresponden a las dos máquinas virtuales y no definen el tamaño de la base de datos administrada ni la capacidad del servicio de almacenamiento de objetos.

Configurar una **red virtual privada**, subredes, reglas de enrutamiento y reglas de firewall que separen el acceso público de la comunicación interna. Web Server será el punto público de acceso a la API y al frontend si se reutiliza uno existente; Worker Server, la cola de mensajería y la base de datos administrada no deberán exponer sus servicios a Internet. Documentar el mecanismo de administración y la conectividad saliente necesaria para instalar dependencias o consumir servicios externos.

Documentación requerida

Arquitectura de la aplicación (10%)

Entregar un documento que explique la solución efectivamente desplegada y los cambios frente a la entrega anterior. Debe incluir:

- **Modelo de componentes:** módulos de la API, workers, cola, base de datos, almacenamiento e integraciones, con sus responsabilidades y comunicaciones síncronas y asíncronas. Incluir el frontend únicamente si se reutiliza uno existente.
- **Modelo de despliegue:** región, red virtual privada, subredes, máquinas virtuales, contenedores, servicio de bases de datos administrado, almacenamiento de objetos, volúmenes persistentes y reglas de acceso relevantes.
- **Decisiones y adaptaciones:** ubicación de Redis o su cola de mensajería, migración al servicio administrado de almacenamiento de objetos del proveedor, configuración de carga directa y URLs firmadas, configuración de los workers y diferencias frente a la arquitectura objetivo del proyecto.
- **Operación y recuperación:** instrucciones de despliegue, migración, verificación, reinicio, respaldo y reconstrucción; ubicación de la configuración y manejo de secretos.
- **Capacidad, costo y limitaciones:** configuración exacta, estimación y consumo observado, puntos únicos de falla y cambios que permitirían evolucionar hacia una aplicación elástica.

Ubicación: directorio `docs/entrega2` del repositorio, referenciado desde `README.md`. Incluir los archivos fuente de los diagramas y los scripts o instrucciones utilizados. Se permite automatizar el aprovisionamiento con infraestructura como código, aunque no se exige introducir una herramienta nueva para esta entrega.

Análisis de capacidad (20%)

El equipo deberá ejecutar los dos escenarios siguientes sobre el entorno desplegado en la nube pública. El objetivo es determinar la capacidad sostenible de la configuración básica, reconocer en qué condiciones comienza a degradarse y explicar con mediciones qué componente limita primero el flujo. No se espera alcanzar una escala de producción ni demostrar los objetivos finales de concurrencia del producto. Tampoco basta con presentar capturas de una herramienta o un máximo de usuarios sin las condiciones en que fue obtenido.

Para cada escenario se combinarán pruebas de carga, orientadas a observar el comportamiento bajo niveles de actividad previstos y crecientes, con una aproximación controlada al estrés, orientada a identificar el punto de degradación o saturación. El alcance termina en caracterizar ese límite y proponer una evolución sustentada en evidencia; no exige redimensionar la infraestructura, implementar escalado automático ni corregir todos los cuellos de botella encontrados.

Condiciones comunes

- Seleccionar una herramienta de generación de carga apropiada para los protocolos y recorridos del escenario. Registrar su nombre y versión, y justificar brevemente la elección por su capacidad para modelar los flujos, automatizar validaciones y exportar resultados reproducibles. No se exige una herramienta específica.
- Utilizar datos sintéticos y scripts reproducibles. Declarar cantidad de usuarios, cursos, recursos, inscripciones e intentos, y las características de los archivos multimedia.
- Ejecutar el generador de carga fuera de las dos máquinas virtuales de la aplicación, registrar su ubicación y recursos, y verificar que no limite los resultados.
- Definir antes de las pruebas la mezcla de operaciones y los endpoints involucrados, la concurrencia o tasa de llegada, el patrón de inyección -constante, incremental o en ráfaga-, el calentamiento, la duración, los incrementos, las pausas entre acciones y el criterio de parada. Los usuarios virtuales deberán recorrer secuencias válidas y no limitarse a repetir un endpoint trivial.
- Establecer una línea base con baja carga y realizar al menos tres niveles crecientes de carga por escenario. Repetir la medición cercana al límite para comprobar su estabilidad. Si el presupuesto impide alcanzar la saturación, reportar el máximo probado y aclarar que no corresponde a la capacidad máxima.
- Registrar p50, p95, p99, throughput, errores y timeouts, junto con CPU, memoria, red y disco de los servidores; conexiones y carga de la base de datos administrada; y profundidad, antigüedad y tasa de procesamiento de la cola. Recoger memoria y disco mediante un agente o herramienta cuando no estén disponibles en las métricas utilizadas.
- Mantener fija la infraestructura y registrar versiones, concurrencia de workers y configuración de caché en cada corrida. Si se realiza un ajuste, presentar el antes y el después por separado.
- Definir éxito y saturación mediante latencias, errores, consumo de recursos y estabilidad de la cola. Distinguir solicitudes válidas fallidas de rechazos esperados por reglas de negocio o límites de tasa, y validar el resultado funcional de las operaciones; un código HTTP exitoso no demuestra por sí solo que el estado esperado se haya conservado.

Escenario 1. Actividad académica concurrente (10%)

Simular estudiantes que consultan el catálogo, acceden a cursos, se inscriben, consultan contenido, registran progreso válido y presentan quizzes. La mezcla debe incluir lecturas y escrituras y utilizar cuentas e intentos diferentes para evitar conflictos artificiales entre usuarios virtuales.

El plan deberá indicar si la autenticación forma parte del recorrido medido o si las sesiones se preparan antes de la corrida. Si se desea evaluar una ráfaga de inicios de sesión, deberá ejecutarse y reportarse como una variante separada para no atribuir su costo a toda la actividad académica. La distribución de operaciones debe representar un recorrido plausible y permanecer constante entre los niveles de carga que se comparen.

El informe debe responder:

- ¿Qué volumen de actividad sostiene la plataforma dentro de los umbrales definidos y en qué nivel comienza la degradación?
- ¿Qué operaciones concentran la latencia o los errores y cómo se relacionan con la API, Redis o su cola de mensajería, el pool de conexiones y PostgreSQL?
- ¿Se conservan la integridad de intentos, la calificación y el progreso bajo concurrencia? Incluir una comprobación de envío duplicado sin doble calificación.
- ¿Qué cambio permitiría aumentar la capacidad y qué medición respalda esa propuesta?

Los scripts deben respetar las reglas de sesión, CSRF, intentos y progreso. Una respuesta HTTP exitosa debe acompañarse de validaciones del resultado esperado. Las peticiones rechazadas por datos o secuencias inválidas no acreditan carga funcional exitosa.

Escenario 2. Carga, procesamiento y consumo multimedia (10%)

Simular profesores que cargan archivos de video y audio directamente al almacenamiento de objetos mientras estudiantes consumen contenido HLS ya disponible desde ese servicio. Utilizar archivos con al menos tres perfiles de duración, tamaño o resolución; declarar los perfiles y las rendiciones esperadas, sin aumentar artificialmente la resolución del original.

Incrementar la tasa de cargas y el consumo concurrente, manteniendo fija la concurrencia de los workers. Las descargas de segmentos deben representar la cadencia de reproducción declarada; descargar todos los segmentos tan rápido como sea posible corresponde a otro patrón de carga y debe identificarse como tal.

El informe debe separar:

- Latencia de la API al autorizar la carga y emitir URLs firmadas, tiempo de transferencia directa al almacenamiento de objetos y tiempo de confirmación de la carga completa.
- Tiempo de espera en cola, duración de procesamiento y tiempo desde la carga completa hasta el estado `available`.
- Trabajos completados por unidad de tiempo, reintentos, fallos y evolución de la profundidad y antigüedad de la cola.
- Latencia, tasa de transferencia y errores de las operaciones sobre el almacenamiento de objetos, consumo de CPU y memoria de los workers y carga de la API al emitir URLs firmadas, confirmar cargas y consultar estados. Distinguir ese tráfico de control de la transferencia de archivos, que ocurre directamente con el almacenamiento.
- Latencia y errores al solicitar manifiestos y segmentos. Si se reporta tiempo hasta el primer cuadro o interrupciones de reproducción, deben medirse con un reproductor; las peticiones HTTP por sí solas no demuestran esas métricas.

Al finalizar la generación de carga, observar el drenaje de la cola y verificar que los trabajos aceptados terminen o queden en un estado de fallo diagnosticable. Un HTTP de aceptación de la carga no equivale a una transcodificación exitosa. El análisis debe identificar qué componente limita el flujo y cómo cambiaría con una CDN, más capacidad de procesamiento o ajustes en la concurrencia de transferencias al almacenamiento de objetos.

Ubicación y formato del reporte

El documento se almacenará en `capacity-planning/pruebas_de_carga_entrega2.md`, con enlaces a los scripts, datos de prueba, resultados originales y gráficas. Debe permitir reconstruir qué se ejecutó y bajo qué condiciones. Como mínimo, incluirá:

- la definición de cada escenario y de sus niveles de carga, con recorridos, endpoints, datos, patrón de inyección, duración, pausas y criterios de éxito, saturación y parada;
- la herramienta y su versión, la infraestructura y configuración efectivas, y las condiciones que se mantuvieron fijas;
- los resultados numéricos por corrida y su variación, acompañados por las métricas de infraestructura y de aplicación necesarias para interpretarlos;
- la identificación del punto de degradación o del máximo probado, el cuello de botella sustentado por evidencia y las limitaciones del experimento;

- una propuesta de evolución relacionada con los hallazgos, indicando qué medición permite esperar que ese cambio aumente la capacidad.

Las gráficas y capturas sirven como evidencia, pero no sustituyen la interpretación. El informe deberá relacionar las métricas del generador con las de Web Server, Worker Server, PostgreSQL, Redis y el almacenamiento de objetos, según el escenario, para explicar qué ocurrió y por qué.

Entregables finales

1. **Plataforma MOOC desplegada en la nube pública**
2. **Release del código y la configuración**
3. **Documento de arquitectura** en `docs/entrega2`
4. **Informe de capacidad** en `capacity-planning/pruebas_de_carga_entrega2.md`, con evidencia reproducible de los dos escenarios.
5. **Video de sustentación**, enlazado desde el README y accesible para el equipo docente.

Toda la entrega se realizará mediante el repositorio del grupo en **GitHub o GitLab**, de acuerdo con el programa del curso. Crear un tag o release identificable, por ejemplo `entrega-2`, y registrar el commit evaluado. El README deberá ofrecer acceso a la documentación, las instrucciones de ejecución, la URL de la aplicación y las evidencias, sin publicar credenciales. El procedimiento de acceso para el equipo docente se comunicará por el medio privado dispuesto en el curso.

Sustentación

Video

Cada grupo preparará un video de **máximo 20 minutos** que presente:

- La arquitectura desplegada y la correspondencia entre sus componentes y los servicios de cómputo, red, almacenamiento y bases de datos del proveedor utilizado.
- Un recorrido de las funcionalidades de la entrega anterior sobre el entorno cloud, mediante clientes HTTP, scripts, pruebas automatizadas o el frontend existente si se reutiliza, mostrando los resultados de la API y los procesos asíncronos.
- Evidencia de carga directa y acceso autorizado al almacenamiento de objetos, procesamiento asíncrono, persistencia en la base de datos administrada, idempotencia y un fallo con reintento.
- Los resultados principales de ambos escenarios de capacidad, el cuello de botella identificado y las propuestas de evolución.

Las pruebas deben emplear diferentes parámetros y archivos multimedia con características variadas. El video podrá mostrar resultados de ejecuciones previas para los procesos largos; los registros completos deberán estar disponibles en el repositorio. Las pruebas automatizadas o una colección ejecutable cubrirán los endpoints del alcance que no puedan recorrerse individualmente dentro del tiempo del video.

Sustentación síncrona

El equipo docente podrá solicitar una sustentación síncrona y la repetición de una prueba funcional o de desempeño. Para ese encuentro, la aplicación deberá estar desplegada y operativa con el servicio administrado de bases de datos relacionales. Si los recursos se eliminaron por control de costos, será responsabilidad del equipo recrearlos previamente mediante el procedimiento documentado.

Todos los integrantes deberán poder explicar las decisiones, la configuración y los resultados. La sustentación es obligatoria en la modalidad indicada por el curso.